

Comparison of WebSocket and HTTP Protocol Performance

Wojciech Paweł Łasocha*, Marcin Badurowicz

Department of Computer Science, Lublin University of Technology, Nadbystrzycka 36B, 20-618 Lublin, Poland

Abstract

The purpose of this article is to compare the performance of the WebSocket and HTTP protocols. For this purpose, LAN equipment and a custom-built testing application were used. It was used to measure the time required to send and download/receive 100-character texts in a specified number of copies, taking into account the speed of laptops and web browsers. The research shows that when transmitting more than 100 copies of data using the WebSocket protocol (compared to HTTP), performance can be increased by several hundred percent. In addition, it has been demonstrated that adding excessive overhead to HTTP requests can slow them down considerably. In contrast, TLS encryption has little effect on the speed of both protocols. It was concluded that the WebSocket protocol is well-suited for sending hundreds or thousands of small data packets per second, whereas for smaller numbers, simple HTTP polling is absolutely sufficient.

Keywords: WebSocket protocol; HTTP protocol; protocol performance comparison

Abstract

The author's goal in this article is to compare the performance of the WebSocket and HTTP protocols. To this end, equipment operating on a LAN and a custom-built testing application were used. Using this application, the time required to send and download/receive 100-character text strings in a specified number of copies was measured, taking into account the speeds of laptops and web browsers. The results of the study show that when transmitting more than 100 copies of data using the WebSocket protocol (compared to HTTP), a performance increase of several hundred percent can be achieved. Furthermore, it was demonstrated that adding excessive overhead to HTTP requests can significantly slow them down. In contrast, TLS encryption has a negligible impact on the speed of both protocols. It was concluded that the WebSocket protocol is well-suited for transmitting hundreds or thousands of small data packets per second, since for smaller numbers, a simple HTTP request is entirely sufficient.

Keywords: WebSocket protocol; HTTP protocol; protocol performance comparison

* Corresponding author

Email address: wojciech.lasocha@pollub.edu.pl (W. P. Łasocha)

©Published under Creative Commons License (CC BY-SA v4.0)

1. Introduction

The HTTP protocol is one of the most important elements of the World Wide Web [1]. It enables data exchange between a client and a server according to the request-response model [2]. According to [3], this was initially sufficient for Internet users, as they were focused on searching for and retrieving information.

As web technologies evolved, there arose a need for greater user interaction on websites. A key milestone in the development of the Web was the introduction of AJAX, which enabled asynchronous communication between a web browser and a web server [4]. However, this technology was still based on the request-response model. Web application developers attempted to use it to emulate real-time communication by employing *HTTP* long polling. Although this method is still widely used across the Internet, it does not solve the problem that data can only be transmitted when a request is initiated by the browser—in this case, it is not possible for the server to send information directly to the client without the client first requesting it. Furthermore, this technique

can cause serious problems due to the heavy load it places on both the client's browser and the server [5].

As explained in articles [6–7], the introduction of the WebSocket mechanism in 2008 marked a breakthrough. It embodies the real-time Web (*Real-Time Web*), which, according to article [8], allows users to receive information immediately after it is published by its authors, rather than requiring them to periodically check the source for updates. This significantly reduces the load on the network, the server, and the browser itself. It avoids additional messages sent and processed on both sides solely to maintain a constant connection and allows the server to send new data to the browser in real time as soon as it appears on the server. These solutions have a positive impact on performance.

According to the author of article [5], executing a single request per connection using the `Socket.IO` WebSocket library is half as fast as with the `HTTP` protocol, because the connection must first be established. In contrast, executing 50 requests within the same connection via WebSocket is half as fast as using

. The same article also states that HTTP reaches a peak throughput of approximately ~950 requests per second, while Socket.IO handles approximately ~3,900 requests per second.

Research conducted by the authors of the article [9] demonstrates that the WebSocket protocol is slightly faster than HTTP. Sending 40,000 bytes of data to a server over a WAN using AJAX took approximately 39 ms, while the WebSocket protocol accomplished the same task in about 29 ms. Sending the same data over a LAN also yielded better results for this technology, being 4 ms faster.

In contrast, the author of the research described in article [10] demonstrated that the use of very fast servers (particularly their processors) allows for handling 5 million simultaneously connected clients using the WebSocket protocol.

The research results in the above articles [5, 9–10] motivate conducting our own research on the performance of both protocols. The author of this article focused on measuring the speed of frequent (repeated) transmission of small data packets, sent in hundreds or even thousands of messages per second. Therefore, the presented results describe how a given protocol performs in applications such as online games, chat rooms, or real-time retrieval of stock quotes. It is unnecessary to consider the case of transferring large files, as the WebSocket protocol is simply not suitable for this purpose due to the fragmentation of data into chunks of several dozen kilobytes each. Therefore, transferring a 1 MB file may take the same amount of time for both HTTP and WebSocket.

2. Research Topic

2.1. Characteristics of the HTTP Protocol

The HTTP protocol establishes a data exchange channel between the two ends of the communication, which are most often a client (such as a web browser) and a server (such as an application running on the computer hosting a website). This protocol is based on a request-response model, i.e., the client sends a request message to the server, after which the server returns a response message containing an HTML file or another resource. The response contains information about the completion status of the request and may also contain the requested content in the message body (for example, the structure of an HTML file). A graphical model of data exchange between the client and the server using the HTTP protocol is shown in Figure 1.



Figure 1: Model of data exchange between a client and a server using the HTTP protocol.

The tasks of an HTTP client include: sending a request—initiating an HTTP connection (via a URL), retrieving a resource from the server, presenting data (building a web page), interacting with the user, caching data from the server, and verifying consistency with the server, as well as encryption and decryption (HTTPS).

In contrast, the tasks of an HTTP server include: handling HTTP requests—executing scripts and sending responses, logging and queuing requests, authentication and access control, encryption and decryption (HTTPS), and selecting the language version of the documents being sent.

HTTP is an application protocol located at Layer 7 of the OSI/ISO model. It is based on TCP (the transport layer protocol) and uses port 80 by default (for unencrypted connections) or port 443 (for encrypted connections). Although it can transmit not only text data but also binary data, it is generally considered a text-based protocol because it uses character-based commands and messages. An important feature of HTTP is that it is a stateless protocol, as it does not retain any information about previous transactions with the client (once a transaction is complete, everything is “forgotten”). The advantages of this approach include better resource utilization (the server-side object handling a request can begin processing the next request immediately after sending the data) and the ease of migrating traffic to other servers. This significantly reduces the server load, but it poses a challenge when, for example, it is necessary to remember a specific state for a user who has previously connected to the server. Web application developers typically solve this problem by using cookies or server-side sessions, or by including hidden parameters (in a form or in the URL).

2.2. Characteristics of the WebSocket Protocol

As described in article [6], WebSocket is a computer communication protocol that provides communication channels via a single TCP connection between a client (such as a web browser) and a server. Importantly, it enables real-time data exchange, meaning the server can send a message to the client without the client first requesting it. This allows for a continuous, two-way conversation between the client and the server while keeping the connection open. Unlike the HTTP protocol, the WebSocket protocol enables full-duplex data transmission, meaning it can send and receive data simultaneously. This protocol has significantly lower overhead and does not send unnecessary messages, as is the case with *HTTP polling*, thereby reducing communication delays, network load, server load, and the load on the web browser itself, which has a positive impact on performance.

The WebSocket protocol is a huge step forward, enabling the creation of real-time, event-driven applications. It can be said that WebSocket offers capabilities that were practically unattainable with the

. Thanks to it, the client does not have to request current data from the server but receives it automatically as soon as it appears on the server. Its potential applications are vast, such as monitoring stock prices, news updates, ticket sales, traffic volume, and readings from medical devices in real time. It should be noted, however, that the HTTP protocol is designed for transmitting text and binary data, whereas the WebSocket protocol is used for the rapid exchange of data in JSON (*JavaScript Object Notation*) format. Web browsers can display a web page based on HTML code. However, there is currently no standard that can do the same by parsing JSON data. For this reason, it can be concluded that the WebSocket protocol serves, in a sense, as an extension to the current Web, rather than a successor to the traditional HTTP protocol.

Before the WebSocket protocol begins exchanging data between the client and the server, the client sends a message to the server in the form of a standard HTTP request asking to switch protocols and establish a persistent connection—this is known as a “handshake.” If the server accepts this request, it sends a response message with a positive acknowledgment to the client. At this point, both parties can exchange data. Of course, the connection can be explicitly closed at any time. If the client or server disconnects, this will always be detected, and the user can be notified of this fact. A graphical model of data exchange between the client and the server using the WebSocket protocol is shown in Figure 2.

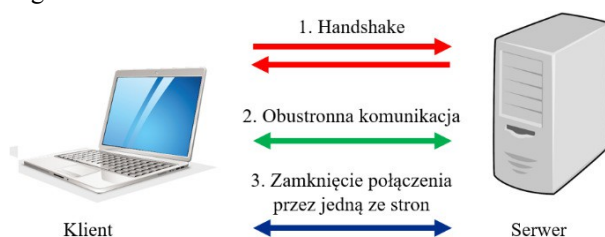


Figure 2: Model of data exchange between the client and the server using the WebSocket protocol.

WebSocket, like the HTTP protocol, is an application-layer protocol located at Layer 7 of the OSI/ISO model, based on TCP (the transport-layer protocol). Although WebSocket differs from HTTP, it is designed to operate over HTTP ports 80 (for unencrypted connections) and 443 (for encrypted connections), and is therefore compatible with HTTP. It is capable of operating over existing infrastructure adapted for HTTP transmission—that is, through proxy servers already configured for this purpose—and can pass through firewall settings as long as HTTP traffic is permitted.

3. Research Methodology

To conduct research on the performance of the HTTP and WebSocket protocols, the author of this article decided

to build his own testing environment. This system consists of the following components:

- Infrastructure – the physical part of the system consisting of a server, an unmanaged switch, and two client computers connected via a four-pair twisted-pair cable to a local *area network* (LAN).
- Base client software – two separate environments installed on the client computers, comprising the Microsoft Windows 10 and Linux Fedora 33 Workstation operating systems, as well as the Google Chrome, Mozilla Firefox, and Microsoft Edge web browsers.
- Base server software – an environment installed on the server, based on the Debian 10 (Buster) operating system, the Node.js platform, and the Express.js library (which facilitates the development of a web server using the HTTP protocol) and the Socket.IO library (which facilitates the development of applications that communicate via the WebSocket protocol).
- Test application – a custom-built web application (run in a web browser) based on the Node.js platform, divided into server-side and client-side components, which measures the transmission time of data with specific parameters using the HTTP and WebSocket protocols.

3.1. Infrastructure Used

The structure of the wired local area network, including the interconnections between all devices, is shown in Figure 3.

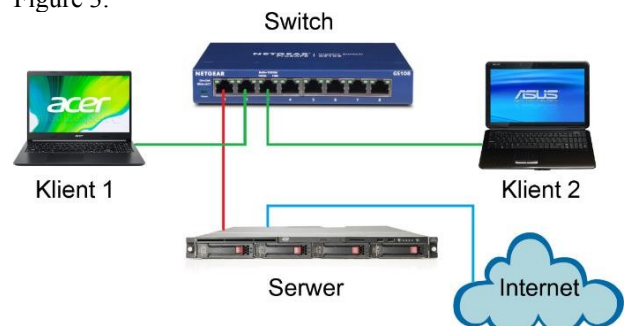


Figure 3: Structure of the wired local area network used in the study.

The central hub connecting the network is a switch. The server is connected to its first port, while the client computers are connected to the second and third ports. The server's second network interface connects to the Internet. The individual devices used have the following specifications:

- Netgear GS108GE Switch
 - 8 Gigabit Ethernet ports
 - Unmanaged
 - No PoE (*Power over Ethernet*)
- HP ProLiant DL320 G5 Server
 - Linux Debian 10 (Buster) operating system
 - 2.4 GHz Intel Xeon Quad-Core processor
 - 6 GB DDR2-800 RAM
 - 4 SAS 15,000 hard drives, 72 GB each
 - 2 Gigabit Ethernet network interfaces

- 1 Acer Aspire 5 laptop
 - Microsoft Windows 10 operating system
 - Intel Core i5-7200U processor, 2.5 GHz
 - 4 GB DDR4-2133 RAM
 - 1000 GB SATA III 5400 hard drive
 - Gigabit Ethernet network interface
- Asus K50IN Laptop 2
 - Operating system: Linux Fedora 33 Workstation
 - Intel Pentium Dual Core T4200 2 GHz processor
 - 4 GB DDR2-800 RAM
 - 60 GB SATA I 5400 hard drive
 - Gigabit Ethernet network interface
- Cabling
 - Four-pair twisted-pair cable
 - Category 6

The server has four hard drives configured in a RAID 0 array. For this reason, we can rule out from the outset that the system's bottleneck was the reading or writing of data sent to or retrieved from the server. The only factor that could have slowed down data transmission was the Asus K50IN client computer (since it is older hardware), which was, of course, taken into account in the tests. However, the use of Gigabit network interfaces in each device should ensure that the test results are as realistic as possible.

3.2. Software Installation and Configuration

To conduct the tests, the author of this article used appropriate software—for both the client computers and the server.

The newer and faster Acer Aspire 5 laptop ran the Microsoft Windows 10 operating system, while the older and slower Asus K50IN ran Linux Fedora 33 Workstation. The former typically comes with the Microsoft Edge web browser pre-installed, while the latter comes with Mozilla Firefox. However, the Google Chrome web browser—which was not included with either operating system—was also used in the tests. Therefore, the missing programs used in the tests had to be installed manually.

The most important software used on the HP ProLiant DL320 G5 server was the Debian 10 (Buster) Linux operating system, which is generally intended for server applications. After its installation, the server's network interfaces were configured so that one network card had Internet access, while the other was accessible on the local network to which the client computers were connected. To save time on network configuration on the clients, a DHCP server was set up on the server. Next, the Node.js platform was installed along with the npm package manager and, additionally, the OpenSSL tool. The latter is used to generate a CSR certificate and a private key, which are required to establish SSL/TLS-encrypted connections (HTTPS and WebSocket Secure).

After completing the above steps, all devices (starting with the server) were restarted.

3.3. Test Application

To install the test application on the server, I had to recursively copy its root directory to the server, then navigate to that directory and run the `npm install` command. Next, it was necessary to modify the contents of the `settings.txt` file located in the `public` directory to specify the IP address of the server's network interface under which it would be recognizable on the LAN to which the client computers were connected.

To run the test application on the server, you had to navigate to its root directory and then, depending on whether TLS encryption was to be used for connections or not, execute one of the following two commands:

- For unencrypted connections:
`node index.js --encryption=no`
- For encrypted connections:
`node index.js --encryption=yes`

From that point on, the application was accessible (from the client side) on all connected clients via web browsers at a URL equal to the server's IP address, preceded by the prefix of the protocol used (`http://` or `https://`), depending on whether data encryption was enabled. Upon opening the web page, the main interface of the test application, shown in Figure 4, appears.

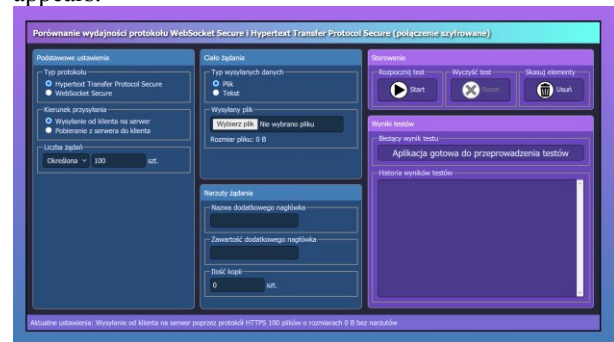


Figure 4: Main user interface of the testing application opened in a web browser.

The user interface of the testing application shown in the figure above consists of two main parts. On the left side (blue boxes), the user can freely customize the settings for the test to be performed. Changes to these settings are displayed in real time in the status bar (located at the bottom of the testing application), which concisely and comprehensively describes the currently selected settings. On the right side of the application interface (purple boxes), the user can control the test and analyze its results once the testing is complete. Selecting test settings involves choosing the protocol type (HTTP/HTTPS or WebSocket/WebSocket Secure), the transfer direction (sending to the server, downloading or receiving from the server, or transfer between clients), the type of data being transferred (file or text) and the number of copies, as well as optional overheads (for the HTTP/HTTPS protocol).

Once the test application has been configured, you can start the test by clicking the **Start** button. The server screen displays real-time progress updates, as shown in Figure 5—in this case, sending 10 texts, each 16 characters long, from the client to the server via the encrypted WSS (*WebSocket Secure*) protocol. The final result of the operation is always displayed in both the client and server applications.

```
10:45:27,852 - Zlecono: Wysylanie od klienta na serwer poprzez protokol WSS
10 Leksliow o dlugosciach 16 znakow
10:45:27,855 - Pomylnie odebrano i zapisano tekst na dysku twardym serwera
(całkowity postep: 10,00%)
10:45:27,857 - Pomylnie odebrano i zapisano tekst na dysku twardym serwera
(całkowity postep: 20,00%)
10:45:27,858 - Pomylnie odebrano i zapisano tekst na dysku twardym serwera
(całkowity postep: 30,00%)
10:45:27,859 - Pomylnie odebrano i zapisano tekst na dysku twardym serwera
(całkowity postep: 40,00%)
10:45:27,861 - Pomylnie odebrano i zapisano tekst na dysku twardym serwera
(całkowity postep: 50,00%)
10:45:27,862 - Pomylnie odebrano i zapisano tekst na dysku twardym serwera
(całkowity postep: 60,00%)
10:45:27,864 - Pomylnie odebrano i zapisano tekst na dysku twardym serwera
(całkowity postep: 70,00%)
10:45:27,866 - Pomylnie odebrano i zapisano tekst na dysku twardym serwera
(całkowity postep: 80,00%)
10:45:27,866 - Pomylnie odebrano i zapisano tekst na dysku twardym serwera
(całkowity postep: 90,00%)
10:45:27,867 - Pomylnie odebrano i zapisano tekst na dysku twardym serwera
(całkowity postep: 100,00%)
10:45:27,867 - Pomylnie ukończono wysylanie tekstow od klienta na serwer (w
yslano 10 tekstow w czasie 0,015 sekund)
```

Figure 5: The Linux console screen on the server after a successful test.

By clicking the **Reset** and **Start** buttons, you can continue testing and analyze the results, and if necessary, change the test parameters. You must also remember to remove from the server any data that was sent to it previously before sending new data to avoid file collisions—this can be done by clicking the **Delete** button. Unfortunately, to enable or disable encryption of the transmitted data, you must restart the application—both on the server side and the client side. You can terminate the server application by pressing the **Ctrl + C** key combination.

4. Results of the Research

4.1. Preliminary Explanations

The research conducted by the author of this article involves testing data transmission via the HTTP and WebSocket protocols, specifically by measuring the upload and download times (or reception times in the case of WebSocket) for short text strings. Each text is approximately 100 bytes in size, as it consists of a string of 100 characters, and its exact content is: 1234567890123456789012345678901234567890123456789012345678901234567890123456789012345678901234567890

Although the text above has a fixed and predetermined size, the tests focus on comparing data transfer times for different numbers of copies, i.e., the number of consecutive request-response pairs (for the HTTP protocol) or messages (for the WebSocket protocol). The measurements included cases for 1, 3, 10, 30, 100, 300, 1,000, and 3,000 copies, and the results are presented in the form of graphs with accuracy to the millisecond (ms).

In the tests, each trial was performed 10 times, and the average value was calculated from these results. The tests took into account the speed of the client computers as well as the performance of the operating systems and web browsers installed on them, since each test was conducted separately for a given client platform.

It should be noted that the tests were conducted using entirely default protocol settings; therefore, much worse results for the HTTP protocol could be obtained if its overhead were increased (as will be demonstrated later in this article), but this is rarely done. At the same time, the WebSocket protocol can sometimes be surprising, as in the tests it took about 90 ms to receive 10 messages from the server, but there were instances where it did so in 15 ms.

4.2. Sending and Receiving via HTTP

The first stage of this study was to examine the transfer speed of 100-character text strings using the HTTP protocol.

First, the time taken to send data from client computers to the server using the POST method was measured. Next, a bar chart was created based on the average values, as shown in Figure 6.

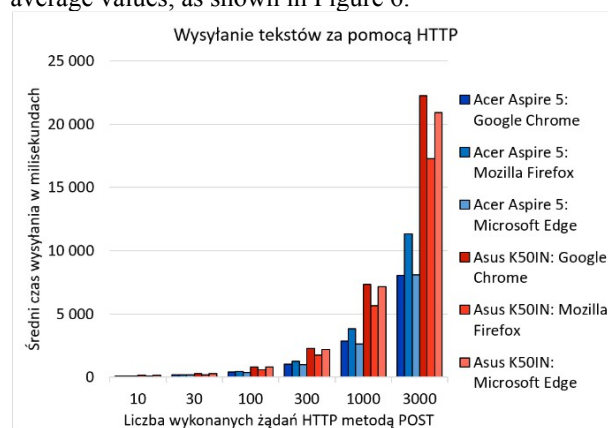


Figure 6: Average times for sending text from client computers to the server using the HTTP protocol.

The chart above clearly shows that executing a larger number of HTTP requests (over 30) using the POST method and processing their responses was faster on the Acer Aspire 5 client computer than on the Asus K50IN, as the former is much newer hardware (and, obviously, faster). Interestingly, there is also a significant difference in the performance of the Mozilla Firefox web browser depending on the platform it runs on (though it is unclear whether this depends on the computer's speed or the operating system: Windows 10 or Fedora 33). However, it's safe to assume that web browsers perform best on the operating systems they come bundled with by default (Edge for Windows and Firefox for Fedora).

In the second step, the time taken to download data from the server to client computers using the GET method was measured. A bar chart based on the average values was then created, as shown in Figure 7.

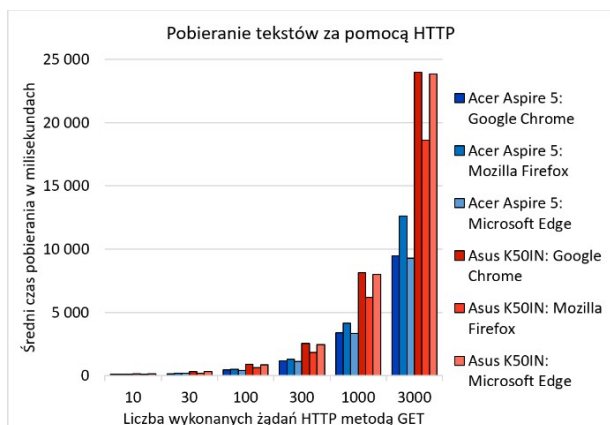


Figure 7: Average times for client computers to download text from the server using the HTTP protocol.

The chart above shows that HTTP GET requests are processed faster on a faster client computer. There was also an increase in the performance of the Mozilla Firefox web browser (compared to other web browsers) depending on the laptop and operating system on which it was used. It can be concluded that Microsoft Edge, regardless of hardware speed, performed just as well as Google Chrome when retrieving data via HTTP GET requests.

4.3. Sending and Receiving via WebSocket

The second stage of this study involved examining the transfer speed of 100-character text strings using the WebSocket protocol.

First, the time taken to send data from client computers to the server was measured. Next, a bar chart was created based on the average values, as shown in Figure 8.

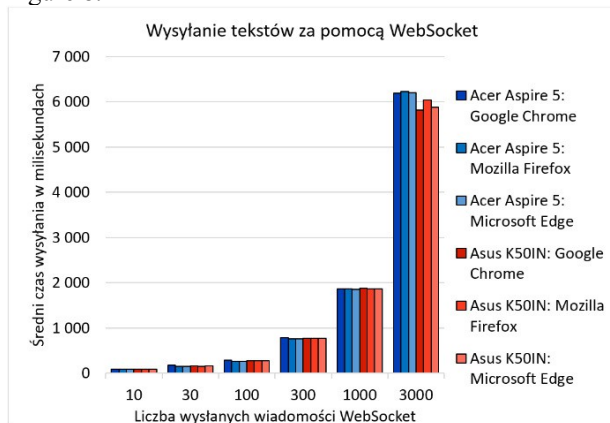


Figure 8: Average times for sending text from client computers to the server using the WebSocket protocol.

What stands out most is the fact that the performance of the WebSocket protocol does not depend on the software used (operating systems and web browsers), nor even on the speed of the client computers. For this reason, developers of applications using the WebSocket protocol do not need to worry about clients

with slow hardware or rarely used software.

In the second step, the time it took for client computers to receive data from the server was measured. Next, a bar chart was created based on the average values, which is shown in Figure 9.

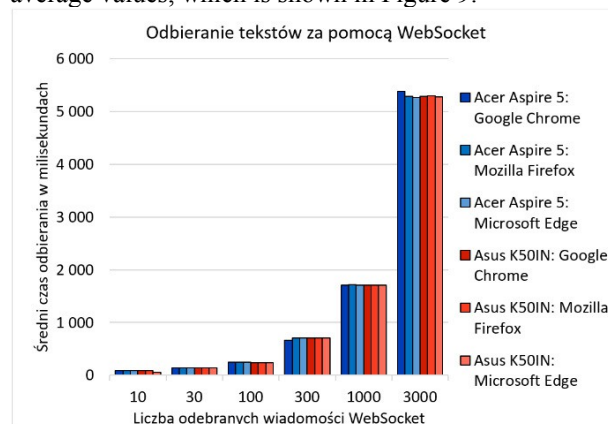


Figure 9: Average times for client computers to receive text from the server using the WebSocket protocol.

The chart above shows that the underlying software (operating systems and web browsers) and client hardware have no impact on the performance of the WebSocket protocol.

4.4. Comparison of HTTP and WebSocket Performance

This section of the article attempts to compare the performance of the HTTP and WebSocket protocols by analyzing their speeds for sending and downloading (receiving, in the case of WebSocket) text. The comparison was based on the test results obtained in the two previous subsections, categorizing them according to the hardware (a faster and a slower laptop) on which they were run, but without taking web browsers into account (i.e., by averaging the results obtained by them). For this purpose, the corresponding bar charts shown in Figures 10 through 11 were created.

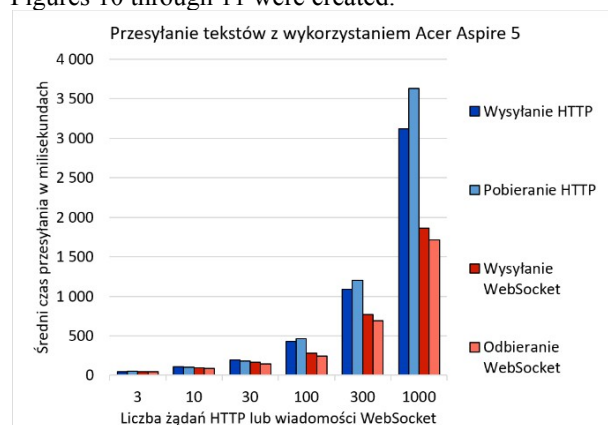


Figure 10: Average text transfer times between the Acer Aspire 5 client computer and the server using the HTTP and WebSocket protocols.

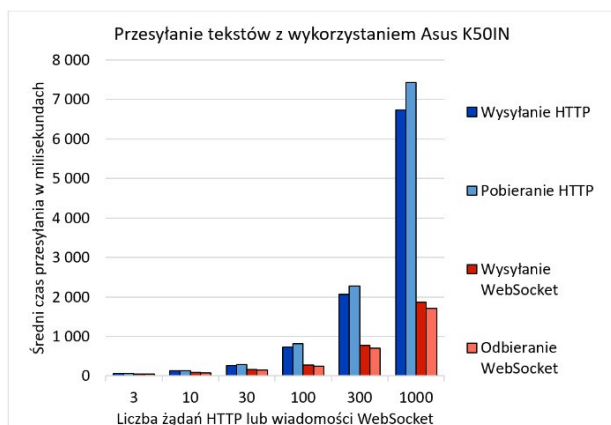


Figure 11: Average text transfer times between the Asus K50IN client computer and the server using the HTTP and WebSocket protocols.

What stands out most when looking at the above graphs is the fact that the WebSocket protocol is significantly faster than HTTP when sending or receiving more than 100 data copies (request-response pairs for HTTP or messages for WebSocket). This difference is particularly pronounced on a slower laptop, since, as noted earlier, the performance of the WebSocket protocol is independent of the client platform. Thus, it can be concluded that there is no point in implementing WebSocket technology to send just a few or a dozen or so messages—in this case, a simple HTTP request is sufficient—especially since computers are becoming faster and faster, handling HTTP requests and responses with increasing efficiency. Thus, the WebSocket protocol is ideal for network applications where it is necessary to transmit large volumes (hundreds or even thousands) of small data packets per second.

It is generally and widely believed that there is no difference in performance between retrieving data from a server using the HTTP GET method and sending it using the POST method. However, research shows that for a larger number of HTTP requests—that is, 100 or more—the latter method is noticeably faster. This is due to the fact that client computers (more specifically, web browsers) are unable to handle such a large number of HTTP GET responses from the server at once.

With the WebSocket protocol, receiving data from the server has always been slightly faster than sending it to the server or forwarding it to other clients. This conclusion is likely influenced by the fact that this technology does not require high performance from the clients themselves but depends on the computing power of the server on which it is used.

Ultimately, by using the WebSocket protocol to transfer a large number of small data packets within a given time frame, performance gains of up to several hundred percent can be achieved. It can even be expected that transferring a few or even a single copy of such data using WebSocket (compared to the standard HTTP protocol) may be slightly faster.

4.5. Impact of Overheads on Transfer Speed

The next stage of this research involved examining the speed of sending 100 copies of data from client computers to the server using the POST method and downloading them from the server using the GET method, along with specific HTTP request overheads. In the tests, each HTTP request (out of the 100 performed) was augmented with a given number of additional header fields in the following format:

name-1: content, name-2: content, etc. The tests included cases with 10, 50, 100, 200, and 300 additional overheads. It should be noted that this technique is only possible with the HTTP protocol—the WebSocket protocol does not allow such operations. Based on the results, a bar chart showing the average values was created, as presented in Figure 12.

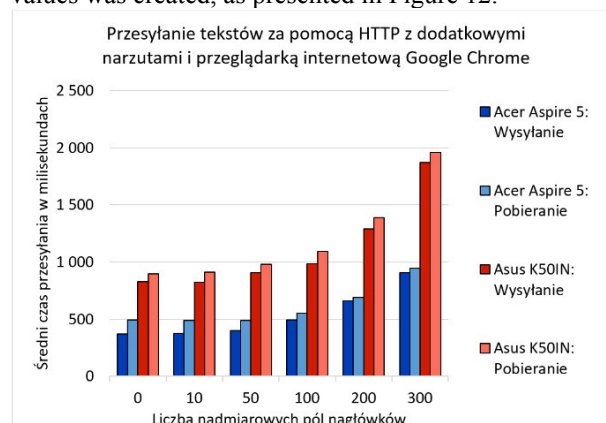


Figure 12: Average transfer times for 100 copies of text between client computers and the server using HTTP with additional overhead.

An analysis of the results shows that adding a few or a dozen additional header fields has no significant impact on the performance of the HTTP protocol. However, if their number reaches 100—which is usually rare—the text transfer speed, according to the conducted tests, may drop by approximately 20%. In this case, the size of the excess overhead in each HTTP request will be approximately 1,900 characters (bytes), hence this result. It should be emphasized that the tests accounted for the addition of extra overhead to each HTTP request; if this overhead were also added to HTTP responses, the result would be even worse.

4.6. The Impact of Encryption on Transfer Speed

The final stage of this study was to examine the transfer speeds of text via the HTTPS protocol (i.e., HTTP with TLS encryption) and the WebSocket Secure protocol (i.e., WebSocket with TLS encryption—WSS for short) and compare them with their plaintext counterparts.

In the first step, the time taken to transfer text between an older, slower client computer and the server was measured using the HTTP and HTTPS protocols, as well as WebSocket and WebSocket Secure,

for a specified number of requests. Based on the results, a bar chart showing the average values was created, as presented in Figure 13.

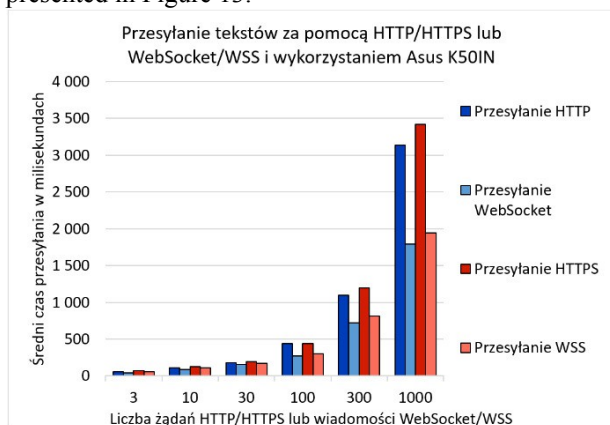


Figure 13: Average text transfer times between the Asus K50IN client computer and the server using the HTTP/HTTPS and WebSocket/WSS protocols.

In the second step, the time taken to transfer text between a newer, faster client computer and the server was measured using the HTTP and HTTPS protocols, as well as WebSocket and WebSocket Secure, for a specified number of requests. Based on the results, a bar chart showing the average values was created, as shown in Figure 14.

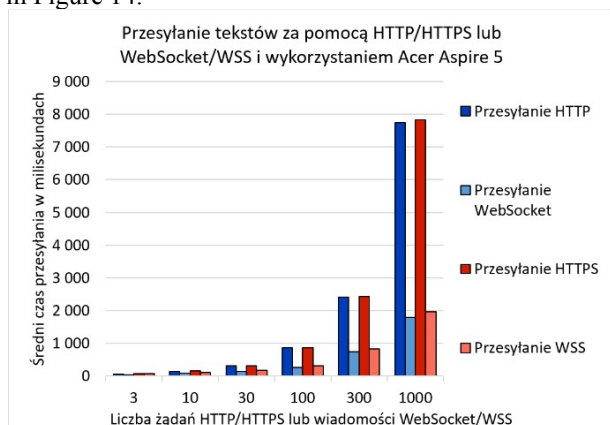


Figure 14: Average text transfer times between the Acer Aspire 5 client computer and the server using the HTTP/HTTPS and WebSocket/WSS protocols

The most important finding from the above charts is that the use of TLS encryption for the HTTP and WebSocket protocols has a minimal negative impact on the time it takes to send data from client computers to the server and to download it from the server to clients, regardless of the client computer's speed. This is due to the low hardware requirements for encrypting and decrypting data.

5. Conclusions

The results of the study show that using the WebSocket protocol, compared to HTTP, can achieve a performance increase of several hundred percent. It can even be predicted that using it to transfer several or a single copy of such data (compared to the usual—

(compared to the standard HTTP protocol) may be slightly faster. This increase is more noticeable for transfers of more than 100 data copies, which is why this new technology is ideal for applications that require the transfer of hundreds or even thousands of data packets per second. The author noted that the tests were conducted using entirely default protocol settings; therefore, if the overhead of HTTP requests or responses increases, the difference will be even greater—a fact that has also been proven. It was also noted that the WebSocket protocol performed equally well on both a slower and a faster laptop, whereas executing HTTP requests and processing their responses performed significantly better on the newer client computer. The use of TLS encryption, for both HTTP and WebSocket, has a negligible impact on the performance of these protocols.

Ultimately, it can be concluded that there is no point in implementing WebSocket technology to transfer a few or a dozen or so data packets, since in this case it is sufficient to use a standard HTTP request—especially since computers are becoming faster and faster and will handle this technique more and more efficiently, not to mention the fact that high-speed Internet is on the horizon, which will eliminate the redundancy and delays generated by this solution.

References

- [1] World Wide Web, in: Wikipedia, The Free Encyclopedia, https://en.wikipedia.org/wiki/World_Wide_Web, January 13, 2021.
- [2] Hypertext Transfer Protocol, in: Wikipedia, The Free Encyclopedia, https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol, [January 13, 2021].
- [3] WebSockets – A Conceptual Deep-Dive, in: Ably Realtime, <https://www.ably.io/concepts/websockets>, [January 13, 2021].
- [4] Ajax (programming), in: Wikipedia, The Free Encyclopedia, [https://en.wikipedia.org/wiki/Ajax_\(programming\)](https://en.wikipedia.org/wiki/Ajax_(programming)), [January 13, 2021].
- [5] WebSocket Simplified, in: Coding Simplified With Shad, <https://iamshadmirza.hashnode.dev/websocket-simplified-cjxjzcu0m002i3hs1eewt2p80>, [January 13, 2021].
- [6] WebSocket, in: Wikipedia, The Free Encyclopedia, <https://en.wikipedia.org/wiki/WebSocket>, [January 13, 2021].
- [7] RFC 6455 – The WebSocket Protocol, in: IETF Tools, <https://tools.ietf.org/html/rfc6455>, [January 13, 2021].
- [8] Real-time web, in: Wikipedia, The Free Encyclopedia, https://en.wikipedia.org/wiki/Real-time_web, January 13, 2021.
- [9] W. Słodziak, Z. Nowak: Performance Analysis of Web Systems Based on XMLHttpRequest, Server-Sent Events, and WebSocket. Springer International Publishing, 2016.
- [10] Benchmark 5-million WebSockets, in: Oat++, <https://oatpp.io/benchmark/websocket/5-million/>, [January 13, 2021].